PHISHXPERT HYBRID

AI-Powered Phishing URL Detection

PROJECT RESULTS REPORT

Project Team Member	Roll Number
Manaswi Ganji	2420030182
Aruna Grace	2420030100

Report purpose

This report summarizes the observed implementation and feature-importance analysis for PhishXpert Hybrid. It records results available from the project logs and distinguishes measured observations from evaluation metrics that still need to be verified.

Prepared from the project information and terminal output shared for this report.

1. Executive Summary

PhishXpert Hybrid is a web-based phishing URL detection project intended to assess submitted URLs using machine-learning techniques. The project context includes engineered URL features, DistilBERT-derived representations, LightGBM classification, and SHAP-based interpretation. The backend is implemented with Python and FastAPI, while the frontend uses React, TypeScript, and Vite.

The terminal output confirms that the FastAPI service started successfully at **http://127.0.0.1:8000**, DistilBERT weights were loaded, and a SHAP feature-importance analysis completed on a dataset of 5,000 rows with 791 features and 500 SHAP samples.

Important evaluation note: the available output does not contain verified test accuracy, precision, recall, F1-score, or a confusion matrix. Those values are therefore not reported as results in this document.

2. Project Objective

The objective is to build an accessible system that accepts a URL and returns a phishing-risk classification. The project aims to combine URL-level signals and learned text representations and to make model behavior easier to inspect through feature-importance analysis.

3. Technology Stack

Component	Technology	Role
Frontend	React, TypeScript, Vite	URL input, API request, result display
Backend API	Python, FastAPI, Uvicorn	Receives requests and runs application logic
Transformer	Hugging Face Transformers, DistilBERT	Provides learned text representations where used by the inference pipeline
Classifier	LightGBM	Tree-based classification / feature analysis
Explainability	SHAP TreeExplainer	Estimates feature contributions to model output

4. Observed Execution Results

Check	Observed result	Interpretation
FastAPI startup	Application startup complete; Uvicorn running on port 8000	The local API process started successfully
Transformer loading	DistilBertModel weights loaded	Model loading completed; loading alone does not prove classifier quality
SHAP analysis input	5,000 rows; 791 features; 500 SHAP samples	The feature-importance analysis ran on the stated data and sample size
SHAP output	Top feature rankings and scores were printed; CSV saved as shap_feature_importance.csv	Feature-importance results were generated

5. SHAP Feature-Importance Results

The following values are transcribed from the terminal output shared for the project. They are SHAP feature-importance scores as printed by the run; they are not percentages and should not be interpreted as standalone probabilities of phishing.

Rank	Feature	SHAP score
1	**external_ratio**	0.700478
2	**url_entropy**	0.559867
3	**has_ip**	0.530444
4	**url_length**	0.529688
5	**https**	0.439553
6	**external_links**	0.390268
7	**num_hyphens**	0.265708
8	**bert_591**	0.253109
9	**num_dots**	0.206715
10	**text_length**	0.203407

Interpretation

In this run, **external_ratio** had the highest reported score (0.700478), followed by **url_entropy** (0.559867), **has_ip** (0.530444), and **url_length** (0.529688). Other highly ranked signals included HTTPS usage, external links, hyphen count, a DistilBERT feature labelled bert_591, dot count, and text length. This indicates that both engineered URL/page characteristics and at least one transformer-derived feature appear in the analyzed feature set. Feature rankings depend on the trained model, data, and SHAP configuration.

The terminal output also listed path_length with a score of 0.171915 in the engineered-feature section. It was not included in the overall top-ten table above because the main top-ten list was preserved as printed.

6. Implementation Notes

The local FastAPI backend responded at port 8000 and its OpenAPI schema was accessible. During frontend integration, the Vite proxy was configured to forward /api/backend requests to port 8001, while the backend was running on port 8000. That mismatch was identified and the proxy target was recommended to be changed to port 8000. The request rewrite removes the /api/backend prefix, so the actual FastAPI prediction route must match the resulting path.

The provided terminal logs confirm successful server startup, but they do not confirm that the frontend prediction request completed successfully end to end. The earlier browser screenshot showed a 502 Bad Gateway response, so the prediction integration should be re-tested before claiming a fully working live demonstration.

7. Evaluation Metrics: Pending Verification

A complete model-results section should include test-set metrics calculated from a held-out dataset. These values were not present in the supplied logs, so they are intentionally marked as pending rather than estimated or fabricated.

Metric	Status	Why it matters
Accuracy	Not provided	Overall fraction of correct predictions
Precision	Not provided	How often predicted phishing cases are actually phishing
Recall	Not provided	How many phishing cases are detected
F1-score	Not provided	Balance between precision and recall
Confusion matrix	Not provided	Counts true/false positives and true/false negatives

8. Limitations and Next Steps

1. End-to-end prediction verification: Confirm that the frontend request reaches the correct FastAPI route and returns a valid prediction.
2. Held-out evaluation: Calculate accuracy, precision, recall, F1-score, and a confusion matrix using a separate test set.
3. Model provenance: Confirm that the inference path loads the intended fine-tuned phishing classifier, not only the base DistilBERT model.
4. RAG/Gemini status: Verify the actual code path before describing retrieval-augmented generation or Gemini-based explanations as active features.
5. Deployment resources: Measure memory and startup behavior before selecting a hosting platform, particularly because the Transformer stack may be resource-intensive.

9. Conclusion

The available project logs demonstrate that the local FastAPI application started, DistilBERT weights loaded, and SHAP feature-importance analysis completed. The reported ranking highlights external_ratio, url_entropy, has_ip, and url_length as the leading features in that analysis. These observations provide concrete implementation evidence for PhishXpert Hybrid. A complete performance claim requires verified held-out evaluation metrics and a successful end-to-end frontend-to-backend prediction test.

Report prepared for project review and hackathon presentation.